

Revisiting AdaIN for Video Style Transfer

NIE Zizheng REN Kaiyan
HKUST
MAIE

zniesag@connect.ust.hk krenab@connect.ust.hk

Abstract

Video style transfer aims to extend image stylization models to continuous sequences while preserving temporal coherence. Although AdaIN provides a flexible and efficient framework for arbitrary image style transfer, its frame-independent formulation inherently leads to temporal flicker when applied to videos. Recent works such as ReReVST mitigate this issue through recurrent refinement, yet these designs introduce architectural complexity and accumulate temporal errors over long sequences. In this work, we revisit the temporal limitations of AdaIN and investigate how far its stability can be improved without altering its architecture. We introduce a lightweight flow-guided temporal loss that leverages optical flow only during fine-tuning to align stylized frame pairs. This approach enhances temporal consistency while maintaining AdaIN’s real-time inference and style generalization capabilities. Experiments demonstrate that our method significantly reduces flicker compared to vanilla AdaIN and offers a practical low-latency alternative to heavier recurrent models and diffusion-based stylization pipelines.

1. Introduction

Image style transfer has achieved remarkable progress due to deep feature representations and perceptual learning. Among existing methods, AdaIN has become a widely adopted baseline for arbitrary style transfer, owing to its simplicity, efficiency, and ability to separate content structure from style statistics. However, AdaIN is inherently designed for image-based inputs and processes each frame independently. When directly applied to videos, this frame-wise formulation produces unstable variations in texture, color, and fine-grained details, resulting in visible temporal flicker—a limitation also observed in our reproduced baseline.

To address temporal instability, recent video style transfer methods introduce explicit temporal propagation modules. ReReVST, for example, employs recurrent refine-

ment and multi-stage updates to enforce coherence across frames. While effective, such architectures come with increased computational cost, potential error accumulation, and reduced suitability for real-time or on-device applications. At the same time, diffusion-based generative models are emerging as powerful tools for high-quality stylization, but their iterative denoising procedures remain too slow for real-time video deployment.

In this work, we revisit the core limitations of AdaIN from the perspective of temporal modeling and investigate whether temporal consistency can be improved without modifying its architecture or sacrificing efficiency. Instead of adding recurrent modules or generative refinement stages, we introduce a lightweight temporal consistency loss applied only during video fine-tuning. Using optical flow to align stylized frame pairs, the decoder learns to produce smoother transitions while preserving all benefits of the original AdaIN framework—arbitrary style generalization, compact architecture, and fast feed-forward inference. Our code and results are available at: <https://github.com/xiao-feizhu/Video-style-transfer.git>

2. Related Work

AdaIN for Image Style Transfer. Adaptive Instance Normalization (AdaIN) [2] provides a simple yet highly effective formulation for feed-forward image style transferring. By matching the channel-wise mean and variance of content features to those of a reference style image, AdaIN decouples style representation from network parameters and enables flexible control such as style interpolation and strength modulation. This flexibility has made AdaIN a widely used baseline for image stylization. However, AdaIN is inherently designed for independent image inputs, and thus when applied to videos in a frame-by-frame manner, the lack of temporal constraints leads to visible flickering. This motivates our work, which retains the original AdaIN structure but enhances its temporal stability through fine-tuning.

Optical Flow Estimation and Frame Alignment. Optical flow provides pixel-level correspondence between consecutive video frames and is a common tool for enforcing temporal consistency in video processing tasks. RAFT [5] is a modern and widely adopted flow estimator that produces accurate and stable motion fields. Because our goal is to introduce temporal smoothness without modifying the AdaIN architecture, we use RAFT only at training time to align stylized results between adjacent frames via simple warping. This allows the model to receive a temporal signal while keeping inference clean and efficient, as no flow computation is required at test time.

Video Style Transfer. Video style transfer aims to extend image stylization models to continuous sequences while avoiding frame-to-frame flicker. Prior works frequently combine stylization with temporal alignment strategies or introduce additional modules that explicitly propagate information across frames. ReReVST [6], for example, enhances video stylization quality through multi-stage refinement and recurrent operations. These approaches typically improve temporal coherence but may require more complicated design choices or additional computation. Our work takes a different perspective: instead of redesigning the stylization pipeline, we keep the core AdaIN framework intact and introduce temporal consistency purely through a training-stage loss. This allows us to evaluate how far a minimal modification can push the temporal performance of AdaIN-based stylization.

3. Method

3.1. Network Architecture

Our approach follows the canonical AdaIN formulation, consisting of a fixed VGG-19 encoder, the AdaIN feature modulation module, and a symmetric decoder responsible for reconstructing stylized images. Importantly, we do not introduce any video-specific modifications to the architecture. This design choice keeps the model simple and ensures that any temporal improvement stems solely from the training procedure rather than from additional components. Fig. 1 shows an overview of our video style transfer network based on AdaIN[2].

3.1.1 Encoder & Decoder

The encoder is the VGG-19 network truncated at layer `relu4_1`, a standard choice in stylization literature due to its ability to extract sufficiently rich perceptual features. The decoder mirrors the encoder’s hierarchical structure using convolution and upsampling layers. During image pre-training, the decoder learns to map the AdaIN-transformed features back to the image space while preserving semantic

content and expressing the target style. In the video fine-tuning stage, the same decoder is refined using temporally related frame pairs, allowing it to learn smoother transitions without changing its overall behavior or structure.

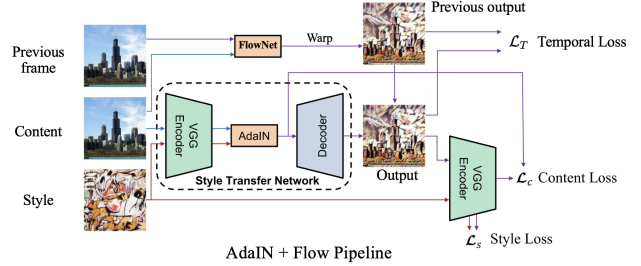


Figure 1. An overview of our video style transfer architecture.

3.1.2 AdaIN

Given content features f_c and style features f_s , the AdaIN operation produces the stylized representation:

$$t = \text{AdaIN}(f_c, f_s), \quad (1)$$

which matches the channel-wise statistics of f_c to those of f_s . This statistic-based alignment allows the decoder to interpret t as the target stylized feature, enabling flexible style manipulation such as interpolation or strength control without retraining the model.

3.2. Loss Functions

Our training objective consists of three components: a content loss, a style loss, and a temporal loss used during video fine-tuning. The overall objective is defined as:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_c + \lambda_s \mathcal{L}_s + \lambda_t \mathcal{L}_t, \quad (2)$$

where λ_s and λ_t control the relative importance of style and temporal consistency, respectively.

3.2.1 Content & Style Losses

We follow the original AdaIN training objective. The content loss encourages the decoder output to stay faithful to the AdaIN-transformed representation by minimizing the discrepancy between their VGG features. The style loss encourages the output to exhibit the statistics of the reference style image, computed on selected VGG layers. These losses ensure that the trained decoder preserves the expected stylization quality before the model is extended to video. Content loss $\mathcal{L}_c$ and style loss $\mathcal{L}_s$ are defined as:

$$\mathcal{L}_c = \|\phi_{l_4}(y) - \phi_{l_4}(t)\|_2^2, \quad (3)$$

$$\mathcal{L}_s = \sum_{i=1}^4 \left(\|\mu(\phi_{l_i}(y)) - \mu(\phi_{l_i}(s))\|_2^2 + \|\sigma(\phi_{l_i}(y)) - \sigma(\phi_{l_i}(s))\|_2^2 \right). \quad (4)$$

3.2.2 Temporal Loss

To introduce temporal coherence, we use optical flow estimated from RAFT to align stylized frames. Given two consecutive content frames (x_t, x_{t+1}) , we first stylize them independently to obtain (y_t, y_{t+1}) . RAFT flow $F_{t \rightarrow t+1}$ is then used to warp y_t toward the next frame:

$$\hat{y}_{t+1} = W(y_t, F_{t \rightarrow t+1}). \quad (5)$$

A simple L_1 difference between $\hat{y}_{t+1}$ and y_{t+1} encourages the stylized outputs to remain consistent across time. This provides temporal guidance during fine-tuning while keeping inference lightweight since no optical flow or warping is needed once training is complete.

4. Experiments

4.1. Datasets

In this work, we utilize two types of datasets that serve different stages of our training pipeline: image style transfer pretraining and video-based fine-tuning. This design enables us to first obtain a strong AdaIN decoder trained on large-scale image pairs, and subsequently adapt it to video style transfer with enhanced temporal behavior.

4.1.1 Image Style Transfer Dataset

For training the base AdaIN image style transfer model, we use a combination of large-scale style and content image datasets:

Content dataset We employ natural images from MS-COCO 2014[3], commonly used in style transfer research due to its diversity in scenes and object categories. This allows the decoder to learn robust and high-quality content reconstruction

Style dataset Artistic images are sourced from WikiArt[1], which contains numerous paintings across a wide variety of artists, styles, and historical periods. The stylistic diversity of WikiArt enables the AdaIN decoder to learn expressive and varied artistic transformations.

During this stage, the encoder is fixed (VGG-19[4]), and only the decoder is trained to reconstruct stylized images. The resulting decoder serves as the initialization for the video fine-tuning stage.

4.1.2 Video Dataset for Fine-tuning

To extend the pretrained decoder to video style transfer, we fine-tune the model using consecutive video frames from the Vimeo-90K triplet[7] dataset. Vimeo-90K provides: high-quality 448×256 resolution video clips, natural motion patterns, clean frame-to-frame correspondence and diverse indoor and outdoor scenes. We follow common practice by using frame pairs (x_t, x_{t+1}) as training samples. We employ optical flow during training to compute frame-to-frame alignment for the temporal loss. Since Vimeo-90K does not provide ground-truth flow, we use only estimated optical flow obtained from a pretrained flow network (e.g., RAFT). The flow is used to warp the previous stylized frame toward the current frame, enabling the temporal loss to measure consistency under motion. Importantly, the optical flow is not used as a supervision target and is not required at test time; it serves solely as an auxiliary alignment cue during fine-tuning.

4.2. Image Style Transfer

Before extending our model to video stylization, we first verify that our re-implemented AdaIN framework reproduces the characteristic behaviors reported in the original work. Using our own decoder trained on COCO and WikiArt, we evaluate three fundamental properties of AdaIN-based image stylization: content–style trade-off, color preservation, and style interpolation. Our results closely match the qualitative behaviors presented in [2], confirming that our implementation serves as a reliable baseline for subsequent video experiments.

Content & style trade-off. Following AdaIN, we modulate the stylization strength using a parameter $\alpha \in [0, 1]$ that linearly interpolates between the AdaIN-transformed feature t from Eq. 1 and the original content feature f_c . The interpolated representation is:

$$\hat{f} = (1 - \alpha) f_c + \alpha t \quad (6)$$

As shown in Fig. 2, varying α produces a smooth transition between the content image and the fully stylized result while preserving the global scene structure.

Color Preservation. We evaluate the color-preserving variant commonly used in style transfer. Given the stylized image y and the content image x , we perform color preservation in the CIE Lab space by replacing the chrominance of y with that of x :

$$y = \text{Lab}^{-1}(L_y, a_x, b_x). \quad (7)$$

This operation preserves the original color palette while maintaining the style-dependent luminance and texture. Representative results are shown in Fig. 3.

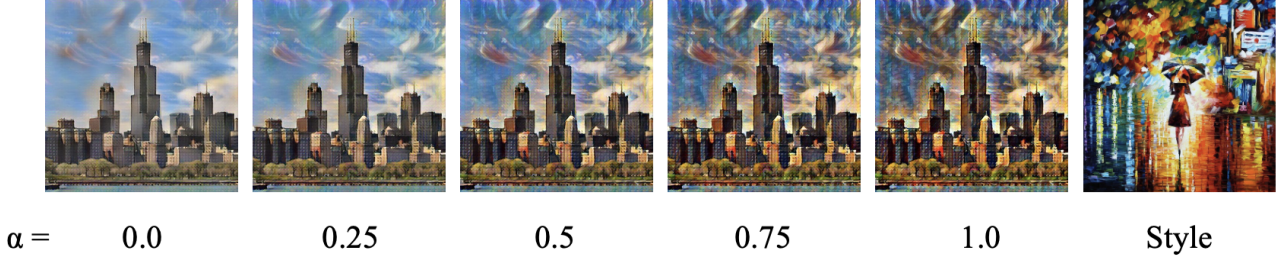


Figure 2. Content-color trade-off.



Figure 3. Color preservation. Left: content and style samples. Right: color-preserved style transfer result.

Style Interpolation Given a content feature F_c and n style features $\{F_{s_i}\}_{i=1}^n$, we obtain the mixed AdaIN feature by a weighted sum:

$$\hat{f} = \sum_{i=1}^n w_i \cdot \text{AdaIN}(f_c, F_{f_i}), \quad (8)$$

where $w_i \geq 0$ and $\sum_{i=1}^n w_i = 1$. As illustrated in Fig. 4, gradually increasing the interpolation weight produces visually coherent transitions between the reference styles.

4.3. Video Style Transfer

After validating the effectiveness of our AdaIN image stylization model, we extend it to the video setting by fine-tuning the decoder on consecutive frames from the Vimeo-90K dataset[7]. Our goal is to improve temporal stability while preserving the visual characteristics of AdaIN stylization.

We train models using several temporal weights to examine their effect on performance. Fig. 5 shows quantitative comparisons.

First, in the total loss curves (left), all models converge smoothly, but higher temporal weights yield a slightly faster reduction during early training. The $T = 4$ model descends more aggressively in the first several thousand iterations, indicating that the temporal term provides a stronger regular-

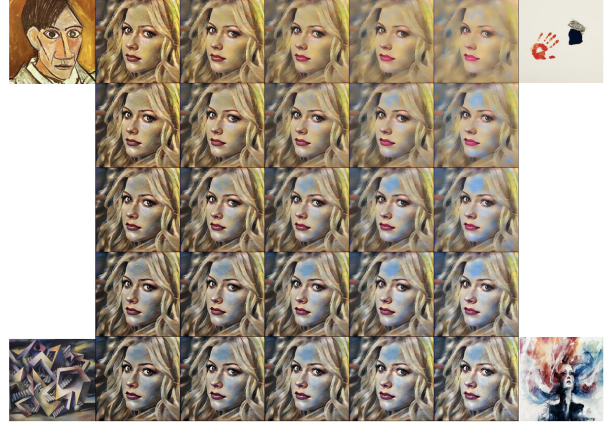


Figure 4. Style interpolation.

ization signal that accelerates decoder adaptation. However, the overall convergence trend remains consistent across the three settings.

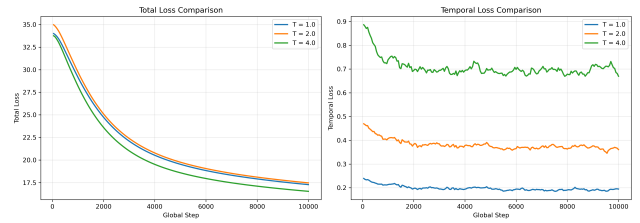


Figure 5. Visualization of fine-tuning result. Left is the total loss curves. Right is the temporal loss curves.

Second, the temporal loss curves (right) exhibit a clear separation aligned with the assigned weights. The $T = 1$ model stabilizes around 0.19, whereas $T = 2$ and $T = 4$ converge to progressively higher baselines. This behavior reflects the fact that larger temporal weights incentivize the model to more strictly match the warped previous frame, which increases the magnitude of the temporal term in the total objective. Despite this difference, all curves reach sta-

ble plateaus early in training, confirming that the temporal branch remains numerically stable under all weights.

Together, these results demonstrate that increasing the temporal weight amplifies the influence of the temporal consistency term without disrupting training stability. Higher weights encourage smoother temporal transitions but may impose stronger constraints on stylization freedom, a trade-off further examined in our qualitative results.

5. Conclusion

In this work, we revisited the inherent limitations of AdaIN for video style transfer, confirming that its frame-wise formulation leads to temporal flicker due to the absence of temporal propagation mechanisms. By introducing a lightweight flow-guided temporal loss during fine-tuning, we achieved significantly smoother frame transitions while keeping the original AdaIN architecture unchanged and fully real-time at inference. Our findings demonstrate that effective temporal stabilization does not require complex recurrent designs; instead, a minimal motion-aligned objective is sufficient to substantially enhance the temporal behavior of feature-transfer-based stylization.

6. Discussion

Building on our observation, ReReVST correctly identifies that temporal cues must be explicitly introduced to constrain the evolution of stylized frames. Its recurrent refinement and multi-stage propagation strategy demonstrates the importance of leveraging historical stylized information. However, these designs also introduce architectural complexity, accumulate errors over long sequences, and incur additional latency that makes real-time deployment more challenging. Our findings suggest that the essence of ReReVST’s success lies in enforcing temporal alignment rather than in its specific recurrent formulation. In this light, incorporating optical flow emerges as a more principled and lightweight way to provide motion-guided temporal supervision. Flow-based alignment explicitly models pixel-level correspondences and helps the model learn stable transitions during training, without the need to introduce heavy recurrent modules or run flow estimation at inference time.

Compared with emerging diffusion-based stylization pipelines, which produce impressive visual quality but require iterative denoising with high computational cost, feature-transfer-based approaches such as AdaIN maintain a compelling advantage in low latency and real-time applicability. While diffusion models dominate high-fidelity image synthesis, they remain expensive for video inference and less suited to applications requiring fast or on-device processing. Our results show that with minimal temporal regularization, AdaIN can be pushed significantly closer to the temporal stability of more complex video styliza-

tion methods while preserving its hallmark strengths—style generalization, structural simplicity, and real-time performance.

References

- [1] Wikiart dataset. <https://www.wikiart.org/>. Accessed: 2025-11-20. 3
- [2] Xun Huang and Serge Belongie. Arbitrary style transfer in real-time with adaptive instance normalization, 2017. 1, 2, 3
- [3] Tsung-Yi Lin, Michael Maire, Serge Belongie, James Hays, Pietro Perona, Deva Ramanan, Piotr Dollár, and C Lawrence Zitnick. Microsoft coco: Common objects in context. In *European Conference on Computer Vision (ECCV)*, pages 740–755, 2014. 3
- [4] Karen Simonyan and Andrew Zisserman. Very deep convolutional networks for large-scale image recognition. *International Conference on Learning Representations (ICLR)*, 2015. 3
- [5] Zachary Teed and Jia Deng. Raft: Recurrent all-pairs field transforms for optical flow. In *European Conference on Computer Vision (ECCV)*, pages 402–419, 2020. 2
- [6] Huan Wang, Zhe Li, Shaodi You, Chang Li, and Yi Yang. Rerevst: Real-time recurrent video style transfer. In *Proceedings of the 29th ACM International Conference on Multimedia*, pages 3633–3642, 2021. 2
- [7] Tianfan Xue, Baian Chen, Jiajun Wu, Donglai Wei, and William T. Freeman. Video enhancement with task-oriented flow. In *International Journal of Computer Vision (IJCV)*, 2019. 3, 4

A. Timeline

10.26 - 10:31: Preparation
11.01 - 11.20: Reproduce AdaIN
11.21 - 11.29: Implement temporal loss

B. Team Contribution

NIE Zizheng: literature review, code implementation, model training and testing, slides preparation and final presentation and report writing.

REN Kaiyan: model training, slides preparation and report writing.